Exercícios — Aula 08

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Crie um projeto novo:

```
npm create vite@latest painel-filmes -- --template react
cd painel-filmes
npm install
npm run dev
```

Faxina obrigatória antes de começar: apague todo o conteúdo de `src/App.jsx` e de `src/App.css`, e delete `src/assets/`. Comece com a tela em branco.

Use estes dados em `src/dados.js`:

```
export const filmes = [
  { id: 1, titulo: "Cidade de Deus",      ano: 2002, genero: "drama",    nota: 8.6,
       assistido: true },
  { id: 2, titulo: "Tropa de Elite",      ano: 2007, genero: "acao",     nota: 8.0,
       assistido: true },
  { id: 3, titulo: "Bacurau",             ano: 2019, genero: "suspense", nota: 7.4,
       assistido: false },
  { id: 4, titulo: "Central do Brasil",   ano: 1998, genero: "drama",    nota: 8.0,
       assistido: false },
  { id: 5, titulo: "Auto da Compadecida", ano: 2000, genero: "comedia",  nota: 8.6,
       assistido: true },
];
```

Nível 1 — obrigatório

Exercício 1 — Primeiro componente

Crie `src/components/Cabecalho.jsx` que recebe `titulo` e `subtitulo` por props e mostra os dois. Use-o no `App.jsx`.

Teste seu entendimento: use o `<Cabecalho />` duas vezes, com textos diferentes. Se funcionou, você entendeu props.

Exercício 2 — Card de filme

Crie `CardFilme.jsx` que recebe um filme por props e mostra título, ano, gênero e nota.

Requisitos:

- Se `assistido` for `true`, mostre um ✅ ao lado do título.

- Se a nota for maior ou igual a 8, adicione a classe `destaque` ao card.
- Estilize minimamente (borda, espaçamento, sombra) para ficar apresentável.

Exercício 3 — Lista de filmes

Crie `ListaDeFilmes.jsx` que recebe o array e renderiza um `CardFilme` para cada um, com `.map()`.

- Use `key={filme.id}`.
- Se a lista estiver vazia, mostre "Nenhum filme encontrado".

Exercício 4 — Estado: marcar como assistido

No `App.jsx`, coloque os filmes em `useState`. Adicione um botão em cada card: **"Marcar como assistido"** / **"Desmarcar"**.

⚠️ **Não mute o array.** Use `map` com spread:

```
setFilmes(filmes.map((f) => (f.id === id ? { ...f, assistido: !f.assistido } : f)));
```

Experimento obrigatório: tente fazer da forma errada primeiro — `filme.assistido = !filme.assistido; setFilmes(filmes);` — e veja a tela não atualizar. Entender esse erro agora vale mais que o exercício inteiro.

Nível 2 — esperado

Exercício 5 — Filtros e busca

Adicione ao `App.jsx`:

1. Botões de filtro: **Todos** / **Assistidos** / **Não assistidos**.
2. Um campo de busca por título (input controlado):

```
const [busca, setBusca] = useState("");

<input value={busca} onChange={(e) => setBusca(e.target.value)} placeholder="Buscar
        filme..." />
```

3. Um `<select>` para filtrar por gênero.

Os três filtros devem funcionar **combinados**.

⚠️ A lista filtrada **não** é um estado novo — ela é *calculada* a partir dos filmes e dos filtros. Regra prática: se dá para derivar do que já existe, não crie um `useState`.

Exercício 6 — Estatísticas

Crie `Estatisticas.jsx`, que recebe a lista e mostra: total de filmes, quantos assistidos, nota média e o filme mais bem avaliado. Tudo calculado com `filter` / `reduce` — a aula 02 de volta.

Exercício 7 — Adicionar filme

Crie `FormularioFilme.jsx` com campos para título, ano e gênero:

```
const [titulo, setTitulo] = useState("");

<form onSubmit={aoEnviar}>
  <input value={titulo} onChange={(e) => setTitulo(e.target.value)} />
  <button type="submit">Adicionar</button>
</form>
```

Requisitos:

- ⚠️ `e.preventDefault()` no `onSubmit`, senão a página recarrega.
- Não deixe adicionar com título vazio — mostre uma mensagem de erro na tela.
- O id do novo filme é o maior id existente + 1.
- Limpe os campos depois de adicionar.
- O formulário **não** guarda a lista: ele chama uma função `aoAdicionar` recebida por props. Quem manda na lista é o `App`.

Desafio — opcional

Exercício 8 — Ordenação e favoritos

1. Um `<select>` para ordenar por: título (A-Z), ano (mais recente) ou nota (maior). Lembre: `[...filmes].sort(...)`, nunca `filmes.sort(...)`.
2. Um botão ⭐ em cada card para favoritar. Os favoritos aparecem sempre no topo da lista.

Exercício 9 — Modo escuro

Adicione um botão que alterna entre tema claro e escuro.

Dica: guarde `const [temaEscuro, setTemaEscuro] = useState(false)` e use isso para trocar a classe do container principal:

```
<div className={temaEscuro ? "app escuro" : "app"}>
```

Depois defina as cores em CSS a partir de `.app.escuro`. Repare como uma única variável de estado muda a aparência inteira da aplicação — sem você tocar em nenhum elemento na mão.

Checklist de entrega

- [ ] Todo componente tem nome começando com letra maiúscula
- [ ] Usei `className`, não `class`
- [ ] Toda lista renderizada tem `key`, e a key é o `id` (não o índice)
- [ ] Nunca alterei um array ou objeto do estado diretamente
- [ ] Não criei estado para coisas que dá para calcular
- [ ] Meus `onClick` passam a função, não o resultado dela
- [ ] Não há avisos vermelhos ou amarelos no console do navegador